



Programação Avançada 2025-26

08 Grafos | ADT Graph

Bruno Silva, Patrícia Macedo

Sumário

- O ADT Graph
 - Especificação;
 - Os tipos (em Java):
 - `Graph<V,E> , e;`
 - `Digraph<V,E> .`
 - Tipos de dados `Vertex<V>` e `Edge<E,V>`
 - Operações disponibilizadas.
 - Exemplo;
 - Exercício autónomo.

ADT Graph | Especificação

- Formalmente, o ADT Graph representa um grafo $G = (V, E)$, composto por um conjunto de vértices V e por um conjunto de arestas E .
- V é um conjunto de vértices, e.g., $\{v1, v2, v3\}$
- E é um conjunto de pares ordenados (v, w) (arestas), onde $\{v, w\} \in V$, e.g., $\{e1 = (v1, v2), e2 = (v1, v3)\}$; cada aresta conecta exatamente 2 vértices.

ADT Graph | Especificação

As operações **modificadoras** do ADT Graph são:

- `insertVertex(elem)` : Cria um novo vértice com o elemento `elem` ;
- `insertEdge(u, v, elem)` : Cria uma nova aresta com o elemento `elem` e que conecta os vértices `u` e `v` . Resulta em erro se `u` ou `v` não forem válidos;
- `removeVertex(v)` : Remove o vértice `v` e todas as suas arestas adjacentes. Resulta em erro se `v` não for válido;
- `removeEdge(e)` : Remove a aresta `e` . Resulta em erro se `e` não for válido. Resulta em erro se `v` não for válido.
- (...)

⚠ Ser "válido" significa pertencer ao grafo existente.

ADT Graph | Especificação

As operações **modificadoras** do ADT Graph são:

- (...)
- `replaceVertex(v, newElem)` : Substitui o elemento armazenado em `v` por `newElem`, devolvendo o antigo. Resulta em erro se `v` não for válido;
- `replaceEdge(e, newElem)` : Substitui o elemento armazenado em `e` por `newElem`, devolvendo o antigo. Resulta em erro se `e` não for válido.

ADT Graph | Especificação

As operações **inspetoras** do ADT Graph são:

- `numVertices()` : Devolve o número de vértices (**ordem** do grafo);
- `numEdges()` : Devolve o número de arestas;
- `vertices()` : Devolve uma coleção dos vértices do grafo;
- `edges()` : Devolve uma coleção das arestas do grafo;
- (...)

ADT Graph | Especificação

As operações **inspetoras** do ADT Graph são:

- (...)
- `opposite(v, e)` : Devolve o vértice da aresta `e` oposto ao vértice `v`. Resulta em erro se `v` ou `e` não existirem no grafo, ou se `v` não é vértice da aresta `e`;
- `incidentEdges(v)` : Devolve a coleção iterável das arestas incidentes ao vértice `v`. Resulta em erro se `v` não existir no grafo;
- `areAdjacent(v, w)` : Devolve um valor lógico que indica se os vértices `v` e `w` são adjacentes. Resulta em erro se `v` ou `w` não existirem como vértices do grafo.

ADT Graph | Java

Da especificação para a linguagem Java 

Os tipos `Graph` e `Digraph`

- `Graph<V,E>` representa um grafo não-orientado;
 - `Digraph<V,E>` representa um grafo orientado (dígrafo)
-

Os tipos genéricos `V` e `E` permitem parameterizar o tipo de elementos a armazenar nos vértices e arestas, respetivamente.

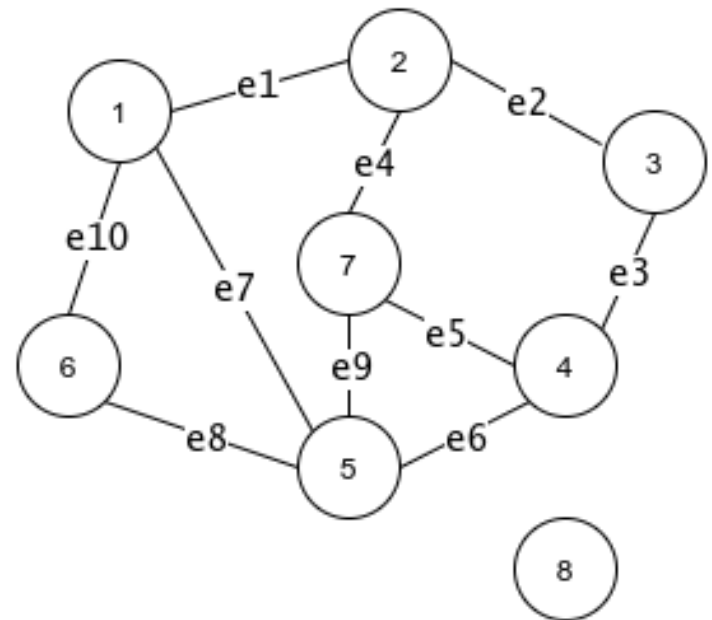
ADT Graph | Java

No caso do grafo ilustrado (não-orientado, rotulado e não valorado), uma instanciação correta seria:

```
Graph<Integer, String> g = new GraphImpl<>();
```

💡 A *interface* `Graph` será apresentada em breve.

⚠️ `GraphImpl` representa uma **qualquer** implementação existente para o tipo `Graph`. O nome concreto da classe poderá ser outro.



ADT Graph | Java

Os tipos `Vertex<V>` e `Edge<E, V>`

- Na perspetiva do utilizador, referências destes tipos permitem identificar vértices e arestas, respetivamente.
- Estes tipos correspondem a *interfaces* e permitem encapsular a representação interna e concreta de um vértice e aresta numa implementação do ADT Graph.
 - Analogamente ao tipo `Position<T>` do ADT Tree.

ADT Graph | Java

A interface **Vertex<V>**:

```
public interface Vertex<V> {  
    public V element();  
}
```

- Na perspetiva do utilizador, para além de poder identificar um determinado vértice do grafo, apenas permite obter o elemento nele armazenado.

ADT Graph | Java

A interface `Edge<E,V>`:

```
public interface Edge<E, V> {  
    public E element();  
    // retorna um array de duas posições: [0] e [1]  
    public Vertex<V>[] vertices();  
}
```

- Na perspectiva do utilizador, para além de poder identificar uma determinada aresta do grafo, apenas permite obter o elemento nele armazenado e que vértices conecta.

ADT Graph | Java

A interface **Graph<V,E>**:

```
public interface Graph<V, E> {  
    int numVertices();  
    int numEdges();  
    Collection<Vertex<V>> vertices();  
    Collection<Edge<E, V>> edges();  
    Collection<Edge<E, V>> incidentEdges(Vertex<V> v)  
        throws InvalidVertexException;  
    Vertex<V> opposite(Vertex<V> v, Edge<E, V> e)  
        throws InvalidVertexException, InvalidEdgeException;  
    boolean areAdjacent(Vertex<V> u, Vertex<V> v)  
        throws InvalidVertexException;  
    Vertex<V> insertVertex(V vElement) throws InvalidVertexException;  
    Edge<E, V> insertEdge(Vertex<V> u, Vertex<V> v, E edgeElement)  
        throws InvalidVertexException, InvalidEdgeException;  
    V removeVertex(Vertex<V> v) throws InvalidVertexException;  
    E removeEdge(Edge<E, V> e) throws InvalidEdgeException;  
    V replace(Vertex<V> v, V newElement) throws InvalidVertexException;  
    E replace(Edge<E, V> e, E newElement) throws InvalidEdgeException;  
}
```

ADT Graph | Java

A interface **Digraph<V,E>**:

```
public interface Digraph<V, E> extends Graph<V, E> {  
    @Override  
    Collection<Edge<E, V>> incidentEdges(Vertex<V> inbound)  
  
    Collection<Edge<E, V>> outboundEdges(Vertex<V> outbound)  
        throws InvalidVertexException;  
  
    @Override  
    boolean areAdjacent(Vertex<V> outbound, Vertex<V> inbound)  
        throws InvalidVertexException;  
  
    @Override  
    Edge<E, V> insertEdge(Vertex<V> outbound, Vertex<V> inbound, E edgeElement)  
        throws InvalidVertexException, InvalidEdgeException;  
  
    @Override  
    Edge<E, V> insertEdge(V outboundElement, V inboundElement, E edgeElement)  
        throws InvalidVertexException, InvalidEdgeException;  
  
}
```

ADT Graph | Java

O *javadoc* destas interfaces pode ser consultado em:

Graph

<https://javadoc.io/doc/com.brunomnsilva/smartgraph/latest/com/brunomnsilva/smartgraph/graph/Graph.html>

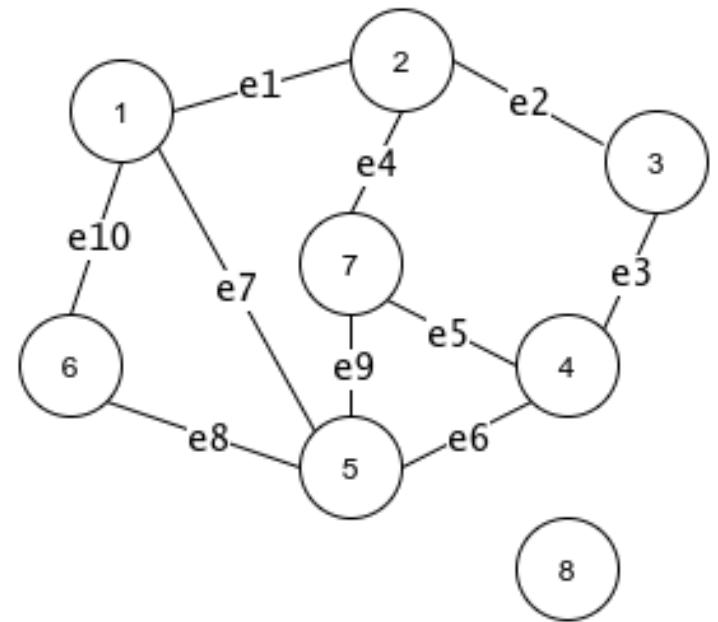
Digraph

<https://javadoc.io/doc/com.brunomnsilva/smartgraph/latest/com/brunomnsilva/smartgraph/graph/Digraph.html>

ADT Graph |

Exemplo

Vamos exemplificar algumas operações, começando pela modelação deste grafo.



ADT Graph | Exemplo

A mera instanciação resulta num grafo vazio:

```
Graph<Integer, String> g = new GraphImpl<>();  
  
System.out.printf("Num. Vertices = %d \n", g.numVertices());  
System.out.printf("Num. Edges = %d \n", g.numEdges());  
System.out.printf("Vertices = %s \n", g.vertices());  
System.out.printf("Edges = %s \n", g.edges());
```

Output:

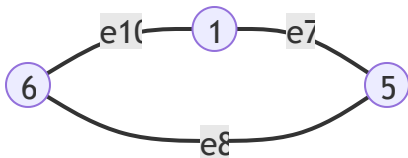
```
Num. Vertices = 0  
Num. Edges = 0  
Vertices = []  
Edges = []
```

ADT Graph | Exemplo

Inserção dos vértices {1, 5, 6} e arestas {e7, e8, e10} :

```
Vertex<Integer> v1 = g.insertVertex(1);  
Vertex<Integer> v5 = g.insertVertex(5);  
Vertex<Integer> v6 = g.insertVertex(6);  
  
Edge<String, Integer> e7 = g.insertEdge(v1, v5, "e7");  
Edge<String, Integer> e8 = g.insertEdge(v6, v5, "e8");  
Edge<String, Integer> e10 = g.insertEdge(v1, v6, "e10");
```

Modelo subjacente resultante:

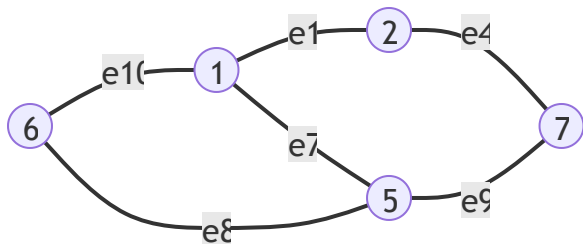


ADT Graph | Exemplo

Inserção dos vértices {2, 7} e arestas {e1, e4, e9} :

```
Vertex<Integer> v2 = g.insertVertex(2);  
Vertex<Integer> v7 = g.insertVertex(7);  
  
Edge<String, Integer> e1 = g.insertEdge(v1, v2, "e1");  
Edge<String, Integer> e4 = g.insertEdge(v2, v7, "e4");  
Edge<String, Integer> e9 = g.insertEdge(v5, v7, "e9");
```

Modelo subjacente resultante:

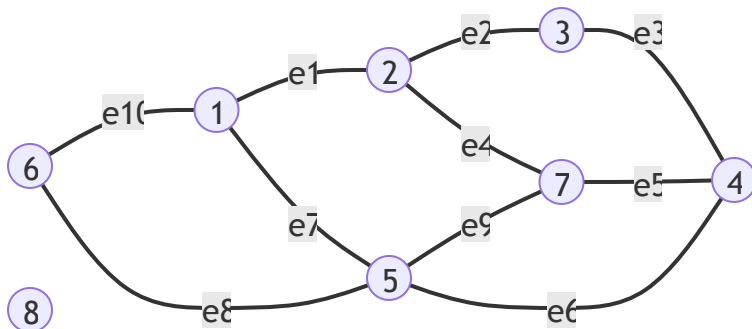


ADT Graph | Exemplo

Inserção dos vértices {3, 4, 8} e arestas {e2, e3, e5, e6} :

```
Vertex<Integer> v3 = g.insertVertex(3);  
Vertex<Integer> v4 = g.insertVertex(4);  
Vertex<Integer> v8 = g.insertVertex(8);  
  
Edge<String, Integer> e2 = g.insertEdge(v2, v3, "e2");  
Edge<String, Integer> e3 = g.insertEdge(v3, v4, "e3");  
Edge<String, Integer> e5 = g.insertEdge(v7, v4, "e5");  
Edge<String, Integer> e6 = g.insertEdge(v5, v4, "e6");
```

Modelo subjacente resultante:



ADT Graph | Exemplo

Se repetirmos a inspeção de informação neste momento:

```
System.out.printf("Num. Vertices = %d \n", g.numVertices());  
System.out.printf("Num. Edges = %d \n", g.numEdges());  
System.out.printf("Vertices = %s \n", g.vertices());  
System.out.printf("Edges = %s \n", g.edges());
```

Output exemplificativo:

```
Num. Vertices = 8  
Num. Edges = 10  
Vertices = [1, 2, 3, 4, 5, 6, 7, 8]  
Edges = [e1, e2, e3, e4, e5, e6, e7, e8, e9, e10]
```

⚠ O *output* das coleções poderá não ser ordenado, na prática.

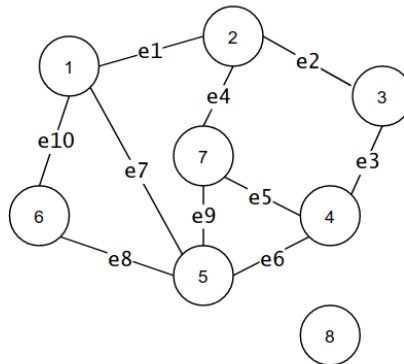
ADT Graph | Exemplo

Para obter as arestas incidentes ao vértice 5, fazemos:

```
Collection<Edge<String, Integer>> incident = g.incidentEdges(v5);  
System.out.printf("Incident edges of 5: (%d) - %s \n",  
    incident.size(), // grau do vértice 5!  
    incident);
```

Output exemplificativo:

```
Incident edges of 5: (4) - [e8, e7, e9, e6]
```



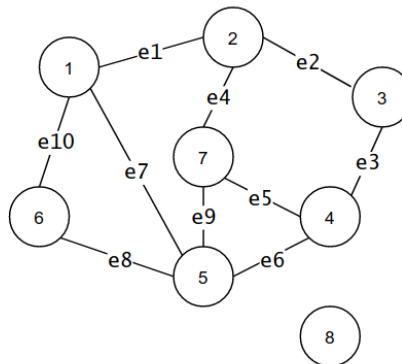
ADT Graph | Exemplo

Exemplos de verificações de adjacências:

```
System.out.println("5 -- 6 ? " + g.areAdjacent(v5, v6));  
System.out.println("5 -- 2 ? " + g.areAdjacent(v5, v2));
```

Output:

```
5 -- 6 ? true  
5 -- 2 ? false
```



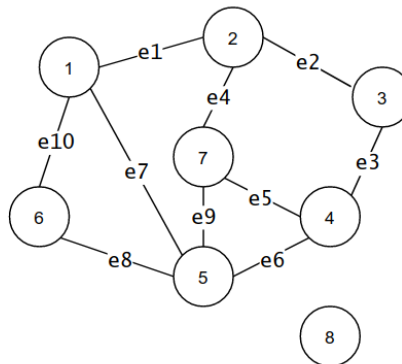
ADT Graph | Exemplo

Obtenção de todos os **vértices adjacentes** a 5 :

```
Collection<Edge<String, Integer>> incident = g.incidentEdges(v5);  
for(Edge<String, Integer> edge : incident) {  
    Vertex<Integer> w = g.opposite(v5, edge);  
    System.out.print(w.element() + " ");  
}
```

Output exemplificativo:

6 1 7 4



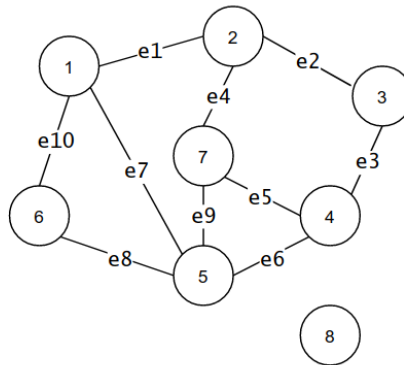
ADT Graph | Exemplo

Tendo uma aresta, saber que vértices conecta:

```
Vertex<Integer>[] vertices = e7.vertices();  
System.out.println(vertices[0].element());  
System.out.println(vertices[1].element());
```

Output exemplificativo:

5
1



ADT Graph | Repositório

O repositório disponível em:

https://github.com/estsetubal-pa/ADTGraph_Example_Template

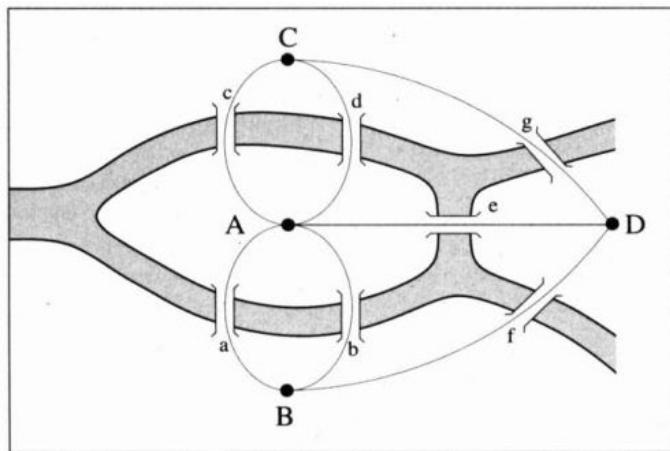
contém o código de modelação do grafo apresentado nestes slides.

Contém como dependência *Maven* a biblioteca JavaFX SmartGraph:

<https://github.com/brunomnsilva/JavaFXSmartGraph>

Treino em casa 🖥️ 💪

1. Clone o repositório anterior e teste o exemplo fornecido.
2. Substitua o modelo original, por uma representação do grafo das pontes de Königsberg:
 - Deverá criar as classes `Local` e `Bridge` para representar o tipo de elementos dos vértices e arestas, respectivamente.



Treino em casa

3. Codifique as instruções necessárias por forma a obter em *output*:

- Todas as pontes que saem do local **A** ;
- Todos os locais vizinhos do local **D** ;
- O número de pontes que partem de **C** ;
- Dados dois locais, verificar se existe uma ponte que os liga diretamente.